



Logistic Regression: Finding the Perfect Decision Boundary

Welcome to an exploration of one of machine learning's most elegant algorithms. Logistic regression combines geometric intuition with mathematical rigor to solve classification problems by finding the optimal decision boundary that separates different classes of data.



What is Classification?

Before: Unlabeled Data

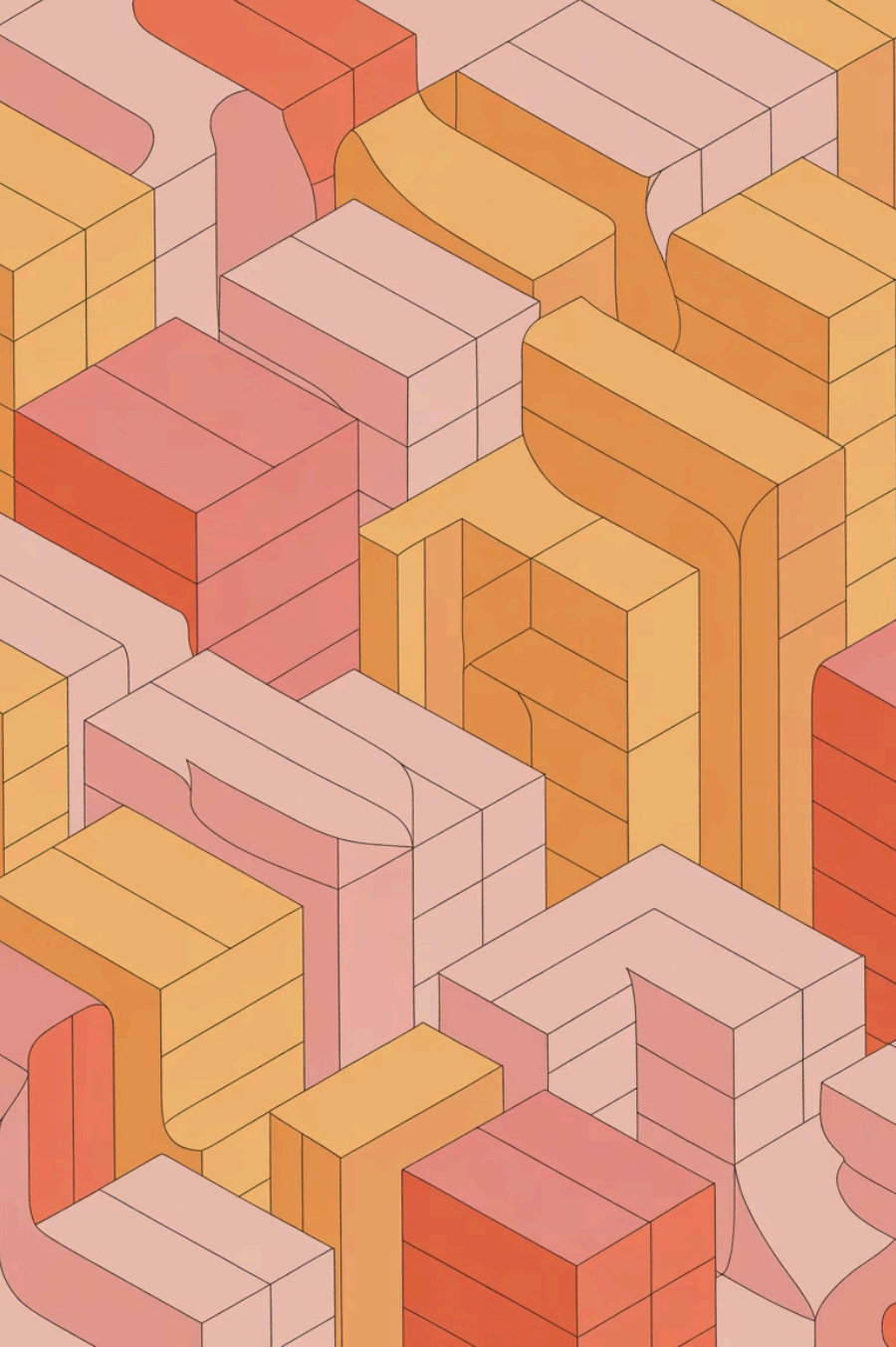
In the real world, we often encounter datasets where observations naturally fall into distinct groups, but we need a systematic way to predict which group a new observation belongs to.

Classification is the process of categorizing data points into predefined classes based on their features. It's fundamental to countless applications, from email spam detection to medical diagnosis.

After: Clear Separation

The goal is to find a mathematical function that can accurately predict the class label for any new observation based on patterns learned from existing data.

Logistic regression achieves this by learning a decision boundary that best separates the classes in the feature space.



The Core Challenge

How do we draw a line (or plane) that best separates two classes when the data points overlap or are close together?

The Problem

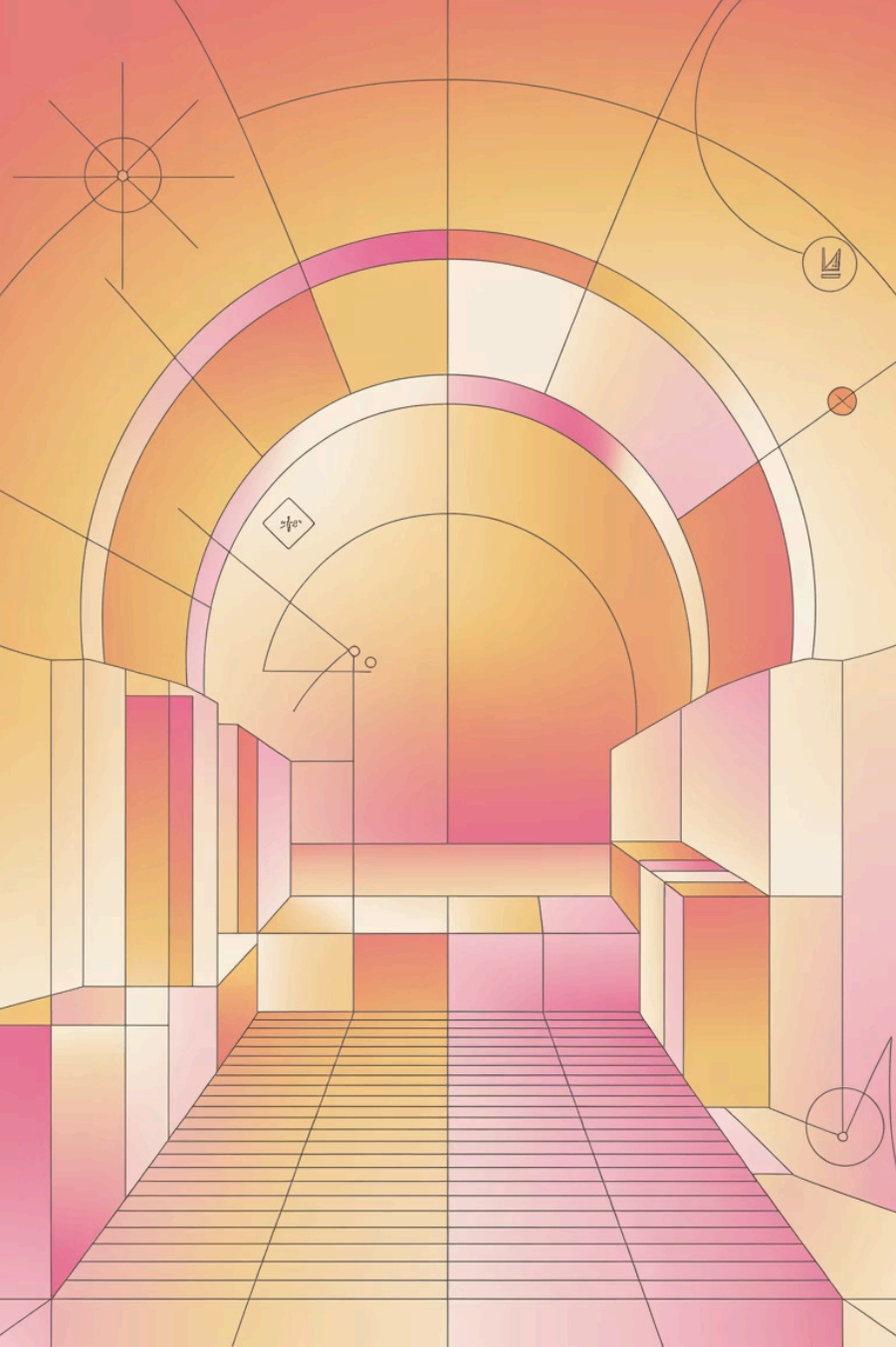
Real-world data is rarely perfectly separable. Classes often overlap in feature space, making it challenging to find a clear boundary.

The Solution

Logistic regression finds the optimal linear decision boundary by maximizing the margin between classes while minimizing classification errors.

The Mathematics

We'll use distance geometry and optimization to formalize this intuitive goal into a precise mathematical objective.

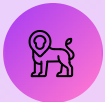


Chapter 1: The Foundation



Key Assumptions of Logistic Regression

Before diving into the mechanics, it's essential to understand the assumptions underlying logistic regression. These assumptions ensure the model works correctly and produces reliable predictions.



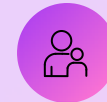
Linear Relationship

There exists a linear relationship between the input features and the log-odds of the outcome. The decision boundary in feature space is a straight line (2D), plane (3D), or hyperplane (higher dimensions).



Independent Observations

Each data point must be independent of others. There should be no hidden correlations between observations that could bias the model's learning process.



No Multicollinearity

Features should not be highly correlated with each other. Multicollinearity can make it difficult to determine the individual effect of each feature on the outcome.

Binary Classification Goal

In binary classification, we have exactly two classes, typically labeled as positive ($y = +1$) and negative ($y = -1$). Our objective is to learn a function that can accurately predict which class a new observation belongs to.



Input: Features

We start with observations described by feature vectors x , where each feature captures a relevant characteristic of the data.



Learn: Decision Function

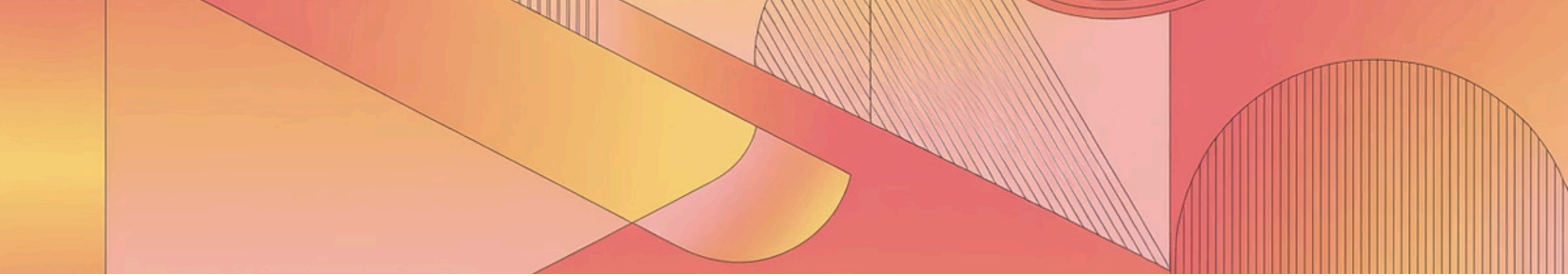
Logistic regression learns weights w that define a decision boundary in the feature space.



Output: Class Labels

The model predicts the class label by determining which side of the boundary a new point falls on.

The beauty of logistic regression lies in its geometric interpretation: we're finding the best separating hyperplane that maximally distinguishes between the two classes.



Chapter 2: The Geometric Intuition

Finding the Best Plane

The central idea of logistic regression is to find a hyperplane (a line in 2D, a plane in 3D, or a higher-dimensional surface) that optimally separates the two classes. This hyperplane is defined by the weight vector $\mathbf{w}$ and is mathematically represented as $\mathbf{wT} \cdot \mathbf{x} = 0$.

01

Define the Hyperplane

The decision boundary is characterized by the equation $\mathbf{wT} \cdot \mathbf{x} = 0$, where $\mathbf{w}$ is the weight vector normal to the plane.

03

Optimize Placement

The optimal hyperplane maximizes the distance (margin) from the boundary to the nearest data points of each class.

02

Position the Boundary

We search for the orientation and position of this hyperplane that best separates the positive and negative examples.



Maximizing Separation Distance

The key objective in logistic regression is not just to find *any* separating hyperplane, but to find the one that maximizes the margin—the distance between the decision boundary and the closest points from each class.

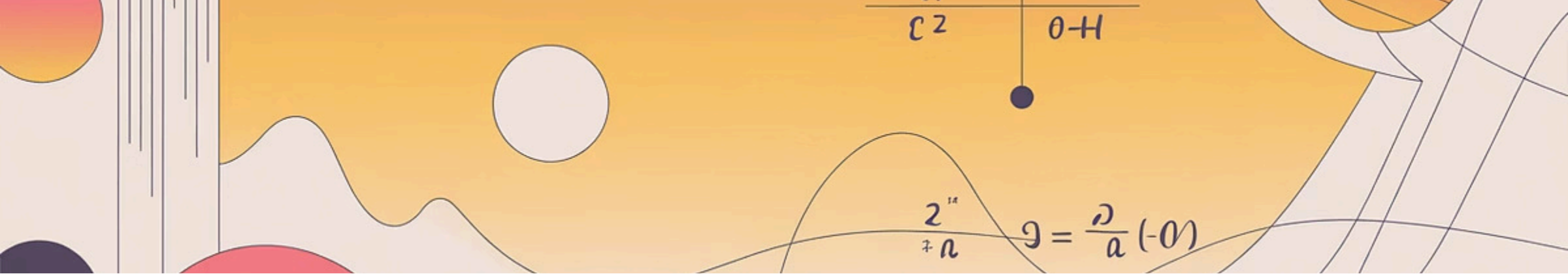
Why Maximize Margin?

A larger margin means the decision boundary is more robust to small perturbations in the data. Points that are far from the boundary are classified with higher confidence, while points near the boundary are more uncertain.

This geometric principle translates into better generalization: a model with a larger margin is less likely to overfit the training data and performs better on unseen examples.

The Margin

The margin is the perpendicular distance from the decision boundary to the nearest data point. By maximizing this distance, we create a "safety zone" around the boundary.



Chapter 3: The Mathematics

Distance Formula: $d = |\mathbf{w}^T \cdot \mathbf{x}| / \|\mathbf{w}\|$

To formalize the concept of margin, we need a mathematical formula for the distance from a point to the decision hyperplane. This distance is given by the perpendicular projection of the point onto the normal vector of the plane.

The Formula

$$d = |\mathbf{w}^T \cdot \mathbf{x}_i| / \|\mathbf{w}\|$$

This measures the perpendicular distance from point $\mathbf{x}_i$ to the hyperplane defined by $\mathbf{w}$.

Numerator: $\mathbf{w}^T \cdot \mathbf{x}_i$

The dot product projects the point onto the weight vector. Its magnitude indicates how far the point is from the hyperplane along the normal direction.

Denominator: $\|\mathbf{w}\|$

The norm of $\mathbf{w}$ normalizes the distance. Without this, the distance would depend on the arbitrary scale of $\mathbf{w}$.

This formula is fundamental to understanding how logistic regression evaluates the quality of a decision boundary. The larger the distances for correctly classified points, the better the boundary.

Same Side vs Opposite Side

The sign of $\mathbf{wT}\cdot\mathbf{x}$ tells us which side of the hyperplane a point lies on. This is crucial for understanding how the classifier makes predictions and how we measure classification correctness.

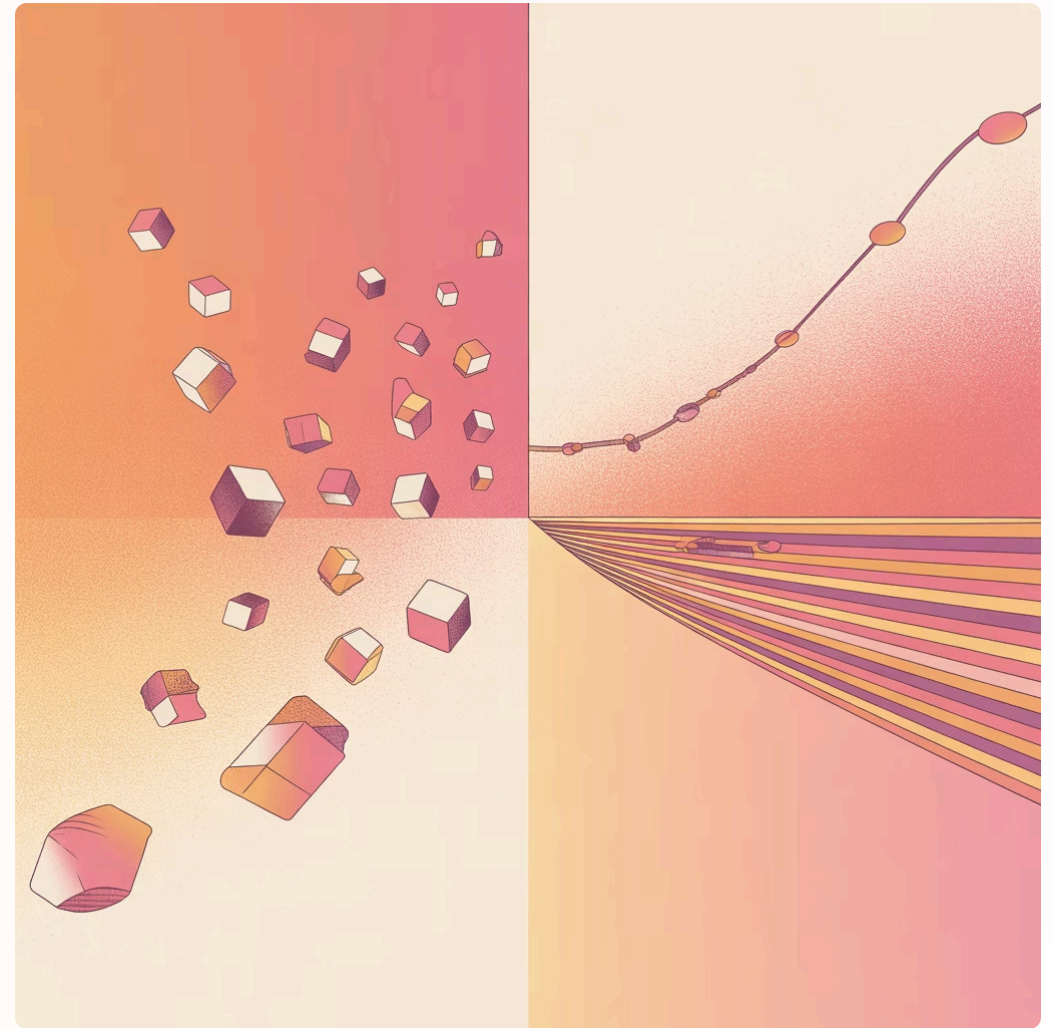
Same Side as $\mathbf{w}$ (Positive)



When $\mathbf{wT}\cdot\mathbf{x} > 0$, the point is on the same side of the hyperplane as the direction of the weight vector $\mathbf{w}$. We classify these points as the positive class.

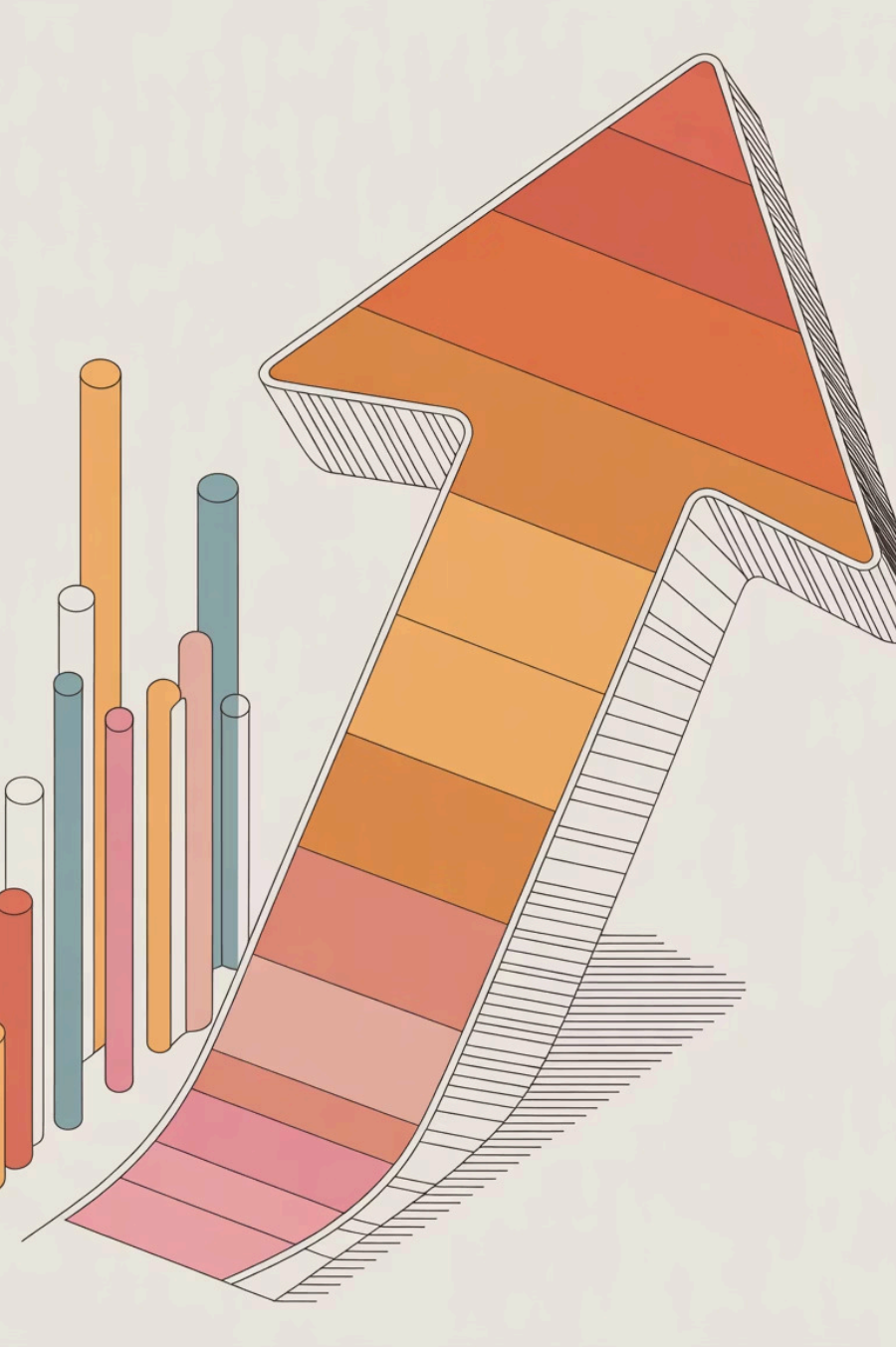
The magnitude indicates confidence: larger positive values mean the point is far from the boundary on the positive side.

Opposite Side (Negative)



When $\mathbf{wT}\cdot\mathbf{x} < 0$, the point is on the opposite side of the hyperplane from the weight vector. We classify these points as the negative class.

Similarly, larger negative magnitudes indicate points that are confidently classified as negative.



The Weight Vector Direction

The weight vector $\mathbf{w}$ is not just a set of coefficients—it has a clear geometric interpretation. It points in the direction of the positive class and is perpendicular (normal) to the decision boundary.

1

Normal to Boundary

The weight vector $\mathbf{w}$ is always perpendicular to the decision hyperplane. This is because the hyperplane is defined by all points $\mathbf{x}$ where $\mathbf{w}^T \mathbf{x} = 0$.

2

Points Toward Positive Class

By convention, $\mathbf{w}$ points in the direction where we expect to find positive class examples. Moving along $\mathbf{w}$ increases $\mathbf{w}^T \mathbf{x}$.

3

Magnitude Matters

The length $\|\mathbf{w}\|$ affects the margin. During training, we often normalize or regularize $\mathbf{w}$ to control model complexity.



Chapter 4: Making Decisions

Classification Rule

With the geometric framework established, we can now formalize the classification rule. The decision function is simple: compute $\mathbf{wT} \cdot \mathbf{x}$ and check its sign.

Positive Class If $\mathbf{wT} \cdot \mathbf{x} > 0$ Predict $y = +1$ The point lies on the positive side of the hyperplane, in the direction of the weight vector.	Decision Boundary If $\mathbf{wT} \cdot \mathbf{x} = 0$ The point lies exactly on the hyperplane. This is the threshold between classes.	Negative Class If $\mathbf{wT} \cdot \mathbf{x} < 0$ Predict $y = -1$ The point lies on the negative side of the hyperplane, opposite to the weight vector direction.
--	---	--

This simple rule, based on the sign of a linear combination of features, is what makes logistic regression both interpretable and computationally efficient.

Correct vs Incorrect Classification

When we have labeled training data, we can evaluate whether each prediction matches the true label. This gives us four possible outcomes, forming the foundation of classification performance metrics.

1

True Positive (TP)

Actual class: $y = +1$, Predicted: $+1$. The model correctly identifies a positive example. These are the best outcomes—correct classifications of the positive class.

2

True Negative (TN)

Actual class: $y = -1$, Predicted: -1 . The model correctly identifies a negative example. Another correct classification, but for the negative class.

3

False Positive (FP)

Actual class: $y = -1$, Predicted: $+1$. The model incorrectly predicts positive when the true class is negative. Also called a "Type I error."

4

False Negative (FN)

Actual class: $y = +1$, Predicted: -1 . The model incorrectly predicts negative when the true class is positive. Also called a "Type II error."

The $y \cdot w^T \cdot x$ Test

There's an elegant mathematical trick to check if a classification is correct: multiply the true label y by the decision function $w^T \cdot x$. The sign of this product tells us everything we need to know.

Correct Classification

When $y \cdot w^T \cdot x > 0$

This means the sign of $w^T \cdot x$ matches the sign of y . The model prediction agrees with the true label.

- If $y = +1$ and $w^T \cdot x > 0$: True Positive
- If $y = -1$ and $w^T \cdot x < 0$: True Negative

The magnitude indicates confidence in the correct prediction.

Misclassification

When $y \cdot w^T \cdot x < 0$

This means the sign of $w^T \cdot x$ opposes the sign of y . The model prediction disagrees with the true label.

- If $y = +1$ and $w^T \cdot x < 0$: False Negative
- If $y = -1$ and $w^T \cdot x > 0$: False Positive

The magnitude indicates how badly misclassified the point is.

 **Key Insight:** The quantity $y \cdot w^T \cdot x$ is called the "functional margin." Maximizing this margin across all training examples is the core objective of many classification algorithms, including logistic regression.

Putting It All Together

Let's synthesize everything we've learned into a complete picture of how logistic regression works geometrically and mathematically.

01

Start with Data

We have training examples $\{(x_i, y_i)\}$ where x_i are feature vectors and $y_i \in \{-1, +1\}$ are class labels.

02

Find Weight Vector w

Learn the weight vector w that defines a hyperplane maximizing the margin—the distance from the boundary to the nearest points.

03

Compute Distance

For each point, calculate $d = |w^T x_i| / \|w\|$ to measure its distance from the decision boundary.

04

Make Predictions

Classify new points by computing $w^T x$ and checking its sign. Positive means class +1, negative means class -1.

05

Verify Correctness

For training data, check if $y \cdot w^T x > 0$. If so, the point is correctly classified; otherwise, it's misclassified.

Working Example: Finding Optimal Weights

Let's work through a concrete numerical example using a small dataset to see how different weight vectors affect the decision boundary, margin, and classification errors.

Section 1: The Dataset

We have 6 data points in 2D space (x1, x2), labeled as +1 (positive class) or -1 (negative class).

0	1.0	2.0	+1
1	2.0	3.0	+1
2	3.0	3.0	+1
3	1.0	1.0	-1
4	2.0	1.0	-1
5	3.0	2.0	-1

Section 2: Testing Different Weight Vectors

1	2	3
Weight Vector 1: $w = [1, 0]$ Predictions: Point (1,2): $1 \times 1 + 0 \times 2 = 1 \rightarrow$ predict +1 ✓ (correct) Point (1,1): $1 \times 1 + 0 \times 1 = 1 \rightarrow$ predict +1 ✗ (wrong! true label is -1) Errors: 3 misclassifications Minimum distance to boundary: 1.0 Comment: Poor choice - doesn't separate classes well.	Weight Vector 2: $w = [0, 1]$ Predictions: Point (1,2): $0 \times 1 + 1 \times 2 = 2 \rightarrow$ predict +1 ✓ Point (2,1): $0 \times 2 + 1 \times 1 = 1 \rightarrow$ predict +1 ✗ (wrong! true label is -1) Errors: 2 misclassifications Minimum distance to boundary: 1.0 Comment: Better, but still has errors.	Weight Vector 3: $w = [-1, 1]$ (Optimal) Decision boundary: $-x_1 + x_2 = 0$, or $x_2 = x_1$ Predictions: (1,2): $-1 + 2 = 1 \rightarrow$ +1 ✓ (2,3): $-2 + 3 = 1 \rightarrow$ +1 ✓ (3,3): $-3 + 3 = 0 \rightarrow$ +1 ✓ (1,1): $-1 + 1 = 0 \rightarrow$ -1 ✓ (2,1): $-2 + 1 = -1 \rightarrow$ -1 ✓ (3,2): $-3 + 2 = -1 \rightarrow$ -1 ✓ Errors: 0 misclassifications! Minimum distance: $\ w\ = \sqrt{2}$, so $d_{\min} = 1/\sqrt{2} \approx 0.71$ Comment: Perfect separation with maximum margin.

Section 3: Key Insights

- Random weights rarely work well – optimization is essential to find the best decision boundary.
- Larger margins mean more confident and robust predictions, as points are further from the classification edge.
- The optimization formula balances the desire for a large margin with the need to minimize training errors.

Real-World Application

To make these concepts concrete, let's consider a medical diagnosis example where logistic regression is used to classify patients as healthy or having a disease based on clinical measurements.

The Scenario

Doctors measure two biomarkers from blood tests. Patients are labeled as healthy ($y = -1$) or having a disease ($y = +1$).

We train logistic regression on historical patient data to learn the decision boundary.

How It Works

- **Feature Space:** Each patient is a point in 2D space where axes represent biomarker levels
- **Decision Boundary:** A line that best separates healthy from disease patients
- **Distance Interpretation:** Points far from the boundary are high-confidence predictions
- **New Patient:** Plot their biomarker values and check which side of the line they fall on
- **Clinical Decision:** Use the prediction to guide further testing or treatment

This example illustrates how the geometric intuition translates directly into practical applications. The distance from the boundary can inform doctors about diagnostic confidence, with borderline cases requiring additional testing.

The Power of Linear Separation

Logistic regression embodies the elegance of machine learning: a simple geometric principle—finding the best separating hyperplane—combined with rigorous mathematical optimization. This approach provides both intuitive understanding and precise predictions.



Geometric Clarity

The visual interpretation as a separating hyperplane makes the model transparent and interpretable, crucial for applications requiring explainability.



Mathematical Rigor

The distance formula $d = |wT \cdot x| / ||w||$ and the margin maximization principle provide a solid theoretical foundation.



Computational Efficiency

Classification requires only a dot product and sign check, making logistic regression fast and scalable to large datasets.

"In many ways, logistic regression represents the perfect balance between simplicity and power—complex enough to solve real problems, yet simple enough to understand deeply."



The Optimization Problem

Now that we understand the geometry and decision rule, we can formalize the optimization problem that logistic regression solves to find the optimal weight vector w .

$$\operatorname{argmax}_w \sum_{i=1}^n y_i \cdot w^T \cdot x_i$$

Subject to constraint:

$$y_i \cdot w^T \cdot x_i > 0 \text{ for all } i$$

What this means:

- We want to find the weight vector w that maximizes the sum of $y_i \cdot w^T \cdot x_i$ across all training points
- The constraint ensures all points are correctly classified (positive product means correct classification)
- This is the foundational optimization problem for linear classification

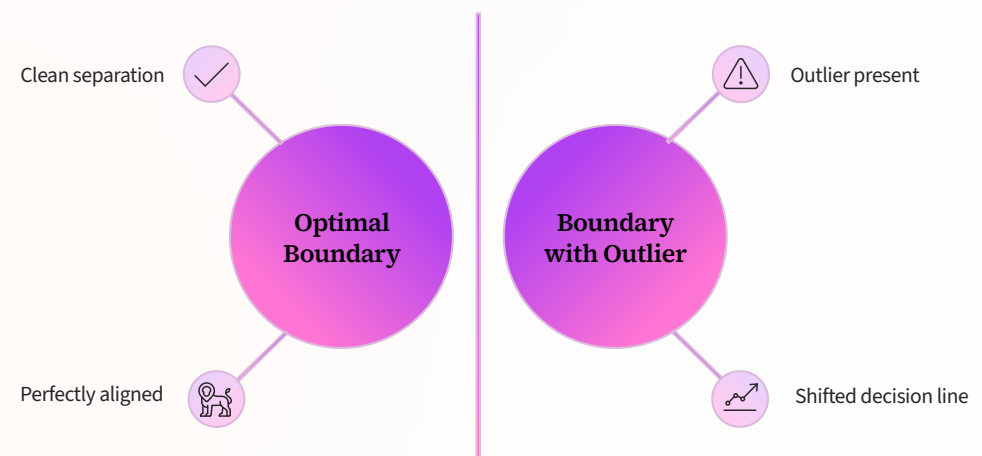
📌 Note: In practice, we add regularization terms and use different loss functions (like sigmoid) to improve generalization and handle non-separable cases. We'll explore these refinements later.

The Outlier Problem

The current optimization approach, which aims to maximize the sum of $y \cdot w^T \cdot x$ across all training points, is highly sensitive to outliers. While effective for perfectly separable data, real-world datasets often contain noise or mislabeled points.

Consider an example: if most points are clustered and separable by a clear boundary, but one outlier data point is located very far away from this cluster, the optimization algorithm will attempt to maximize the distance to that single, distant outlier. This happens because the objective function includes the contribution of every point's distance, giving disproportionate weight to extreme values.

This can result in a suboptimal decision boundary that doesn't effectively maximize the margin for the majority of points. The boundary shifts to accommodate the outlier, compromising the overall separation quality for the bulk of the data.



This illustrates why simply maximizing raw distances can be problematic, especially when data contains noise or extreme values. Robust classification models need ways to mitigate the influence of such outliers.

Why We Need the Sigmoid Function

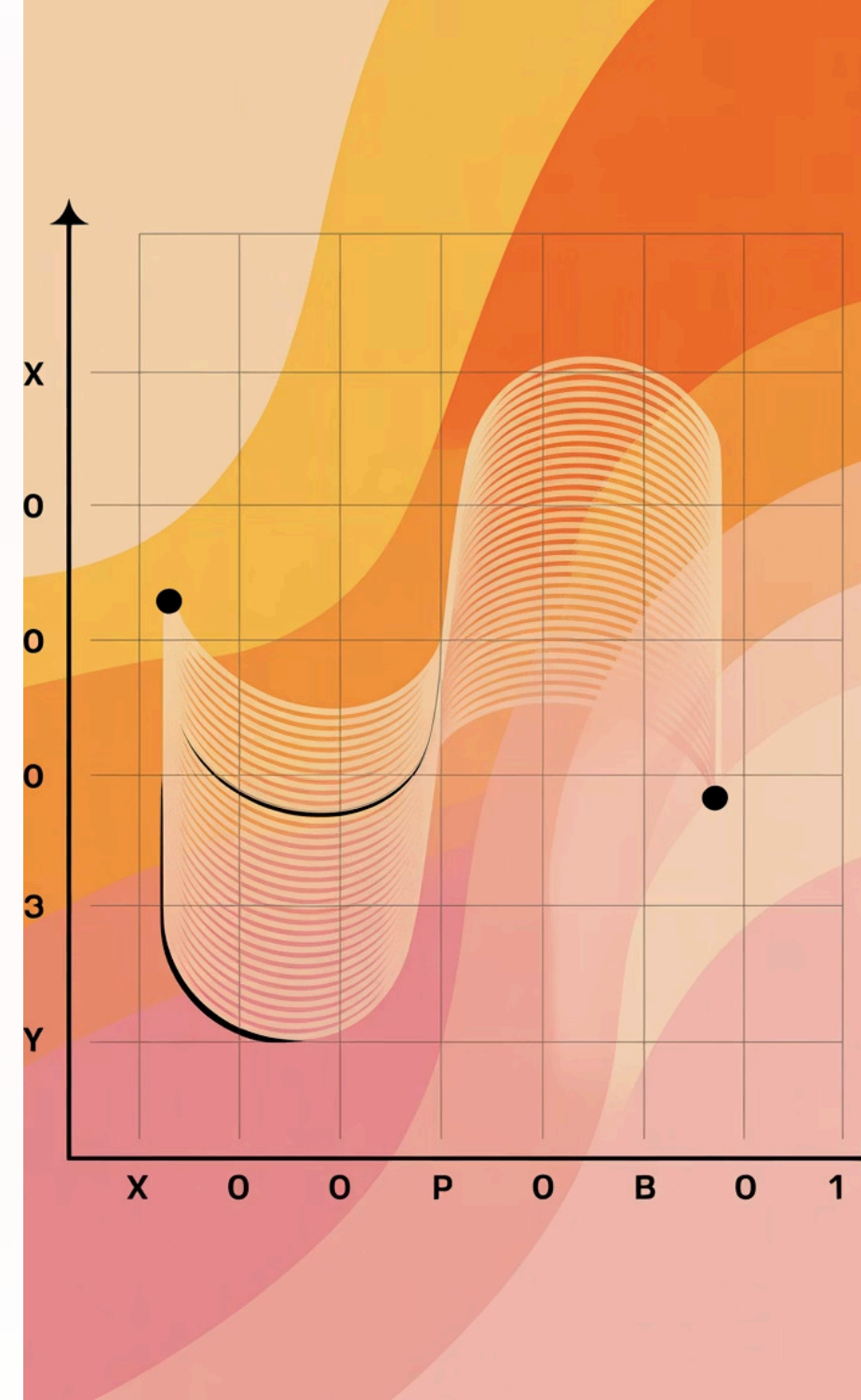
The current optimization approach, which directly uses raw distances like $y \cdot w^T \cdot x$, produces values that can range from $-\infty$ to $+\infty$. This unbounded nature makes the model extremely sensitive to outliers, as a single far-off point can drastically alter the decision boundary by contributing a very large or very small value to the objective function.

To mitigate this problem and transform these unbounded values into meaningful probabilities between 0 and 1, logistic regression employs the sigmoid function, $\sigma(z) = 1/(1 + e^{-z})$. This function "squashes" the extreme outputs of $w^T \cdot x$ into a predictable range, making the model more robust to noisy data.

● Key properties of sigmoid:

- Maps large positive values $\rightarrow$ close to 1
- Maps large negative values $\rightarrow$ close to 0
- Maps 0 $\rightarrow$ exactly 0.5 (decision boundary)
- Smooth, differentiable curve (important for optimization)

The sigmoid "squashes" extreme values, effectively reducing the influence of outliers by mapping them to values closer to 0 or 1 without allowing them to dominate the scale. Instead of maximizing raw distances, we now work with bounded probability values, where the output can be directly interpreted as the probability of belonging to the positive class.



The Complete Optimization Journey

The journey to the robust logistic regression objective function involves a series of transformations, each designed to solve specific problems encountered in the previous step.

01

Raw Distance Maximization

$$\max \sum (y \cdot w^T \cdot x)$$

Problem: Unbounded, sensitive to outliers

02

Sigmoid Transformation

$$\sigma(w^T \cdot x) = \frac{1}{1 + e^{-w^T \cdot x}}$$

Solution: Bounded probabilities [0,1]

03

Likelihood Function

$$\prod \sigma(y_i \cdot w^T \cdot x_i)$$

Problem: Numerical underflow

04

Log-Likelihood

$$\max \sum \log(\sigma(y_i \cdot w^T \cdot x_i))$$

Solution: Stable, convex optimization

This systematic evolution ensures that the final objective function is robust, numerically stable, and theoretically sound, forming the foundation of modern logistic regression.



Regularization: Preventing Overfitting

Even with log-likelihood, the model can still overfit to training data

Overfitting occurs when the model learns noise and specific patterns in training data rather than general patterns

This leads to poor generalization on new, unseen data

Regularization adds a penalty term to the objective function to constrain the weights

Two common types:

L2 Regularization (Ridge)

Penalizes large weights, adds $\lambda ||w||^2$ term

L1 Regularization (Lasso)

Encourages sparsity, adds $\lambda ||w||_1$ term

The regularized objective function becomes:

$$\max \sum \log(\sigma(y_i \cdot w^T \cdot x_i)) - \lambda \cdot R(w)$$

where $R(w)$ is the regularization term and λ controls the strength

Benefits:

- Prevents weights from becoming too large
- Improves generalization to new data
- Reduces model complexity
- Makes the solution more stable

The regularization parameter λ balances fitting the training data vs keeping weights small